



ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



Chương 3

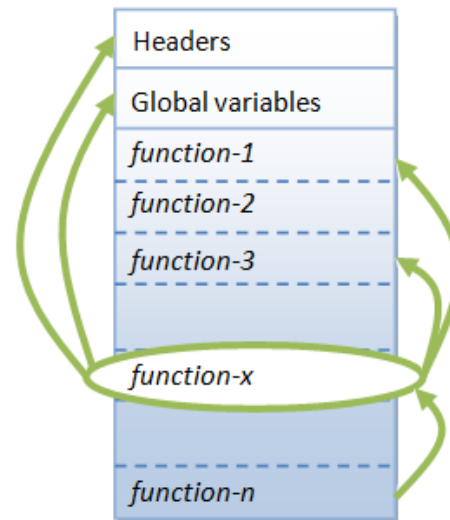
JAVA OOP

Giảng viên: ThS. Nguyễn Thành Luân



Nhắc lại

- ❖ Ngôn ngữ lập trình hướng thủ tục truyền thống (chẳng hạn như C, Fortran, Cobol và Pascal)



A function (in C) is not well-encapsulated

Nhắc lại

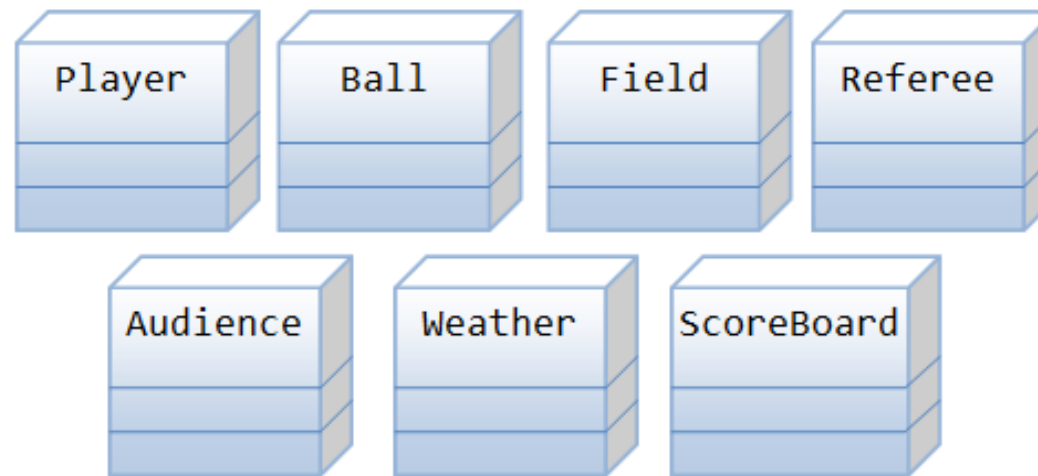
- ❖ Ngôn ngữ lập trình hướng thủ tục truyền thống (chẳng hạn như C, Fortran, Cobol và Pascal)
 - ✓ Khó sao chép một hàm từ một chương trình và sử dụng lại trong một chương trình khác.
 - ✓ Không có tính trừu tượng cao để giải quyết các vấn đề thực tế trong cuộc sống.
 - ✓ Tách biệt cấu trúc dữ liệu (biến) và thuật toán (hàm).

Tại sao cần dùng OOP?

- ❖ Các ngôn ngữ lập trình hướng đối tượng (OOP) được thiết kế để khắc phục những vấn đề này.
- ❖ Tính đóng gói tốt.
- ❖ Tính trừu tượng cao, phù hợp để giải quyết các vấn đề thực tế trong cuộc sống.

Tại sao cần dùng OOP?

❖ Ví dụ:



Classes (Entities) in a Computer Soccer Game

Tại sao cần dùng OOP?

❖ Các lợi ích của OOP

- ✓ *Dễ dàng thiết kế phần mềm*
- ✓ *Dễ dàng bảo trì phần mềm*
- ✓ *Phần mềm có thể tái sử dụng*

OOP trong Java

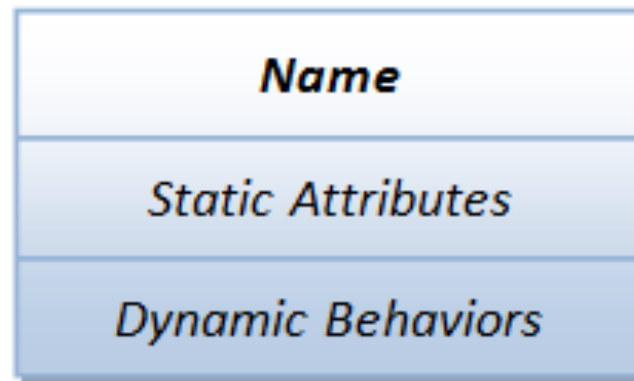
Tính thống nhất

Tính thống nhất

Lớp và Thể hiện của lớp (Class & Instances)

- ❖ Một lớp (Class) là một định nghĩa của các đối tượng cùng loại.
- ❖ Một thể hiện (Instance) là một trường hợp cụ thể của một lớp.
- ❖ Ví dụ: Ta có thể định nghĩa một lớp "Student" và tạo ra ba thể hiện (instance) của "Student" cho "Peter", "Paul" và "Pauline".
- ❖ Lưu ý: Thuật ngữ “object (đối tượng)” thường dùng để chỉ instance.

Lớp và Thể hiện của lớp (tt)



A class is a 3-compartment box

Lớp và Thể hiện của lớp (tt)

❖ Ví dụ về các lớp

Name (Identifier)	Student	Circle	SoccerPlayer	Car
Variables (Static attributes)	name gpa	radius color	name number xLocation yLocation	plateNumber xLocation yLocation speed
Methods (Dynamic behaviors)	getName() setGpa()	getRadius() getArea()	run() jump() kickBall()	move() park() accelerate()

Examples of classes

Lớp và Thể hiện của lớp (tt)

❖ Ví dụ về các thể hiện của lớp

Name	<u>paul:Student</u>	<u>peter:Student</u>
Variables	name="Paul Lee" gpa=3.5	name="Peter Tan" gpa=3.9
Methods	getName() setGpa()	getName() setGpa()

Two instances - paul and peter - of the class Student

Định nghĩa lớp

```
[AccessControlModifier] class ClassName {  
    // Class body contains members (variables and methods)  
    .....  
}
```

Quy ước đặt tên lớp :

✓Tên lớp là một danh từ hoặc một cụm danh từ.

✓Tất cả các từ sẽ được viết hoa chữ cái đầu tiên (PascalCase).

```
public class Circle {           // class name  
    double radius;             // variables  
    String color;  
  
    double getRadius() { ..... } // methods  
    double getArea() { ..... }  
}
```

```
public class SoccerPlayer {    // class name  
    int number;                // variables  
    String name;  
    int x, y;  
  
    void run() { ..... }      // methods  
    void kickBall() { ..... }  
}
```

Tạo các thể hiện của một lớp

- ❖ Để tạo *một instance của một lớp*, bạn phải:
 - ✓ **Khai báo** tên instance của một lớp cụ thể.
 - ✓ **Khởi tạo** instance (tức là cấp phát bộ nhớ cho instance và khởi tạo instance) bằng cách sử dụng toán tử "new".

Tạo các thể hiện của một lớp (tt)

❖ Ví dụ,

```
// Declare 3 instances of the class Circle, c1, c2, and c3  
Circle c1, c2, c3; // They hold a special value called null  
// Construct the instances via new operator  
c1 = new Circle();  
c2 = new Circle(2.0);  
c3 = new Circle(3.0, "red");  
  
// You can Declare and Construct in the same statement  
Circle c4 = new Circle();
```

Lưu ý: Khi một instance được khai báo nhưng không được khởi tạo, nó giữ một giá trị đặc biệt gọi là null.

Toán tử (.)

- ❖ *Các biến và các phương thức* thuộc một class được gọi là *các biến thành viên* và *các phương thức thành viên*.
- ❖ Để tham chiếu đến một biến hoặc phương thức thành viên, bạn phải:
 - ✓ 1. Xác định instance mà bạn quan tâm, và sau đó,
 - ✓ 2. Sử dụng toán tử (.) để tham chiếu đến biến hoặc phương thức thành viên mong muốn.

Toán tử (.) (tt)

❖ Ví dụ,

```
// Suppose that the class Circle has variables radius and color,  
// and methods getArea() and getRadius().  
// Declare and construct instances c1 and c2 of the class Circle  
Circle c1 = new Circle ();  
Circle c2 = new Circle ();  
// Invoke member methods for the instance c1 via dot operator  
System.out.println(c1.getArea());  
System.out.println(c1.getRadius());  
// Reference member variables for instance c2 via dot operator  
c2.radius = 5.0;  
c2.color = "blue";
```

Các biến thành viên

- ❖ *Một biến thành viên có một tên (hoặc định danh), một kiểu dữ liệu (type) và giữ một giá trị của kiểu dữ liệu mà nó được khai báo.*
- ❖ **Quy ước đặt tên biến:** Tên biến nên là **một danh từ hoặc một cụm danh từ**. Từ đầu tiên viết thường và các từ còn lại viết hoa chữ cái đầu tiên (**camelCase**), ví dụ: `fontSize`, `roomNumber`, `xMax`, `yMin` and `xTopLeft`.
- ❖ **Cú pháp để định nghĩa biến:**

[`phamViTruyCap`] kiểuDữLiệu tenBien [= giáTriBanDau];

Các biến thành viên (tt)

- ❖ Ví dụ,
 - ✓ `private double radius;`
 - ✓ `public int length = 1, width = 1;`

Các phương thức thành viên

❖ Một phương thức:

1. Nhận các tham số,
2. Thực hiện các thao tác được định nghĩa trong thân phương thức, và
3. Trả về một phần của kết quả (hoặc void).

❖ Cú pháp khai báo phương thức như sau:

```
[phamViTruyCap] kieuTraVe tenPhuongThuc ([danhSachThamSo]) {  
    // Phần thân phương thức .....  
}
```

Các phương thức thành viên (tt)

❖ Ví dụ,

```
// Return the area of this Circle instance
public double getArea() {
    return radius * radius * Math.PI;
}
```

❖ Quy ước đặt tên phương thức:

- ✓ Tên phương thức nên là một động từ, hoặc cụm động từ.
- ✓ Từ đầu tiên viết thường và các từ còn lại viết hoa chữ cái đầu tiên (camelCase).
- ✓ Ví dụ: `getArea()`, `setRadius()`, `getParameterValues()`, `hasNext()`.

Tên biến vs Tên phương thức vs Tên lớp?

Tên biến vs Tên phương thức vs Tên lớp?

- ❖ Tên biến là danh từ hoặc cụm danh từ, biểu thị một thuộc tính;
- ❖ Tên phương thức là một động từ hoặc cụm động từ, biểu thị một hành động.
- ❖ Tên lớp là một danh từ hoặc cụm danh từ, biểu thị một đối tượng.

Một ví dụ về OOP

Instances

<u>c1:Circle</u>	<u>c2:Circle</u>	<u>c3:Circle</u>
-radius=2.0 -color="blue"	-radius=2.0 -color="red"	-radius=1.0 -color="red"
+getRadius() +getColor() +getArea()	+getRadius() +getColor() +getArea()	+getRadius() +getColor() +getArea()

Một ví dụ về OOP (tt)

Class Definition

Circle
-radius:double=1.0 -color:String="red"
+getRadius():double +getColor():String +getArea():double

Instances

<u>c1:Circle</u>	<u>c2:Circle</u>	<u>c3:Circle</u>
-radius=2.0 -color="blue"	-radius=2.0 -color="red"	-radius=1.0 -color="red"
+getRadius() +getColor() +getArea()	+getRadius() +getColor() +getArea()	+getRadius() +getColor() +getArea()

Một ví

Circle.java

```
1  /*
2  * The Circle class models a circle with a radius and color.
3  */
4  public class Circle {    // Save as "Circle.java"
5      // Private instance variables
6      private double radius;
7      private String color;
8
9      // Constructors (overloaded)
10     public Circle() {          // 1st Constructor
11         radius = 1.0;
12         color = "red";
13     }
14     public Circle(double r) {   // 2nd Constructor
15         radius = r;
16         color = "red";
17     }
18     public Circle(double r, String c) { // 3rd Constructor
19         radius = r;
20         color = c;
21     }
22
23     // Public methods
24     public double getRadius() {
25         return radius;
26     }
27     public String getColor() {
28         return color;
29     }
30     public double getArea() {
31         return radius * radius * Math.PI;
32     }
33 }
```

Một ví dụ về OOP (tt)

```
1  /*
2   * A Test Driver for the "Circle" class
3   */
4  public class TestCircle {    // Save as "TestCircle.java"
5      public static void main(String[] args) {    // Program entry point
6          // Declare and Construct an instance of the Circle class called c1
7          Circle c1 = new Circle(2.0, "blue"); // Use 3rd constructor
8          System.out.println("The radius is: " + c1.getRadius()); // use dot operator to invoke member methods
9          System.out.println("The color is: " + c1.getColor());
10         System.out.printf("The area is: %.2f%n", c1.getArea());
11
12         // Declare and Construct another instance of the Circle class called c2
13         Circle c2 = new Circle(2.0); // Use 2nd constructor
14         System.out.println("The radius is: " + c2.getRadius());
15         System.out.println("The color is: " + c2.getColor());
16         System.out.printf("The area is: %.2f%n", c2.getArea());
17
18         // Declare and Construct yet another instance of the Circle class called c3
19         Circle c3 = new Circle(); // Use 1st constructor
20         System.out.println("The radius is: " + c3.getRadius());
21         System.out.println("The color is: " + c3.getColor());
22         System.out.printf("The area is: %.2f%n", c3.getArea());
23     }
24 }
```

Một ví dụ về OOP (tt)

Compile `TestCircle.java` into `TestCircle.class`.

Run the `TestCircle` and study the output:

```
The radius is: 2.0  
The color is: blue  
The area is: 12.57  
The radius is: 2.0  
The color is: red  
The area is: 12.57  
The radius is: 1.0  
The color is: red  
The area is: 3.14
```

Phương thức khởi tạo (Constructor)

- ❖ **Constructor:** Là một phương thức đặc biệt dùng để khởi tạo các đối tượng.
- ❖ Ví dụ:
 - ✓ `Circle c1 = new Circle();`
 - ✓ `Circle c2 = new Circle(2.0);`
 - ✓ `Circle c3 = new Circle(3.0, "red");`
- ❖ **Lưu ý:**
 - ✓ Tên của Constructor giống với tên lớp.
 - ✓ Constructor không có kiểu trả về.
 - ✓ Constructor chỉ có thể được gọi thông qua toán tử “new”.
 - ✓ Constructor không được kế thừa (Sẽ tìm hiểu vào ngày hôm sau).
- ❖ **Constructor mặc định:** Một constructor không có tham số được gọi là constructor mặc định.

Nạp chồng phương thức (Overloading)

- ❖ Nạp chồng phương thức (Method Overloading) có nghĩa là *cùng một tên phương thức* có thể có *các cách triển khai* (phiên bản) *khác nhau*.
- ❖ Tuy nhiên, các triển khai khác nhau phải được phân biệt bằng danh sách tham số của chúng (số lượng tham số hoặc loại tham số hoặc thứ tự của chúng).

Nạp chồng phương thức (tt)

```
1  /*
2   * Example to illustrate Method Overloading
3   */
4  public class TestMethodOverloading {
5      public static int average(int n1, int n2) {           // version A
6          System.out.println("Run version A");
7          return (n1+n2)/2;
8      }
9
10     public static double average(double n1, double n2) { // version B
11         System.out.println("Run version B");
12         return (n1+n2)/2;
13     }
14
15     public static int average(int n1, int n2, int n3) {   // version C
16         System.out.println("Run version C");
17         return (n1+n2+n3)/3;
18     }
19
20     public static void main(String[] args) {
21         System.out.println(average(1, 2));               // Use A
22         System.out.println(average(1.0, 2.0));           // Use B
23         System.out.println(average(1, 2, 3));             // Use C
24         System.out.println(average(1.0, 2));             // Use B - int 2 implicitly casted to double 2.0
25         // average(1, 2, 3, 4); // Compilation Error - No matching method
26     }
27 }
```

Nạp chồng Constructor của lớp Circle

- ❖ Circle()
- ❖ Circle(double r)
- ❖ Circle(double r, String c)

Phạm vi truy cập: public vs private

- ❖ ***public***: Các Lớp / biến / phương thức bên trong lớp và **tất cả các đối tượng khác** trong hệ thống đều có thể truy cập.
- ❖ ***private***: Chỉ có các Lớp / biến / phương thức **bên trong lớp** mới có thể truy cập.

Ẩn dấu thông tin và đóng gói

- ❖ Các biến thành viên của một lớp thường được ẩn khỏi từ bên ngoài (tức là các lớp khác), với phạm vi truy cập private.
- ❖ Quyền truy cập vào các biến thành viên được cung cấp thông qua các phương thức truy cập public, ví dụ: `getRadius()` và `getColor()`.
- ❖ **Ẩn dấu?**
- ❖ **Đóng gói?**
- ❖ **Nguyên tắc:** Không sử dụng phạm vi public cho bất kỳ biến nào, trừ khi bạn có lý do chính đáng.

Các public Getters và public Setters cho các biến private

```
// Setter for color
public void setColor(String newColor) {
    color = newColor;
}

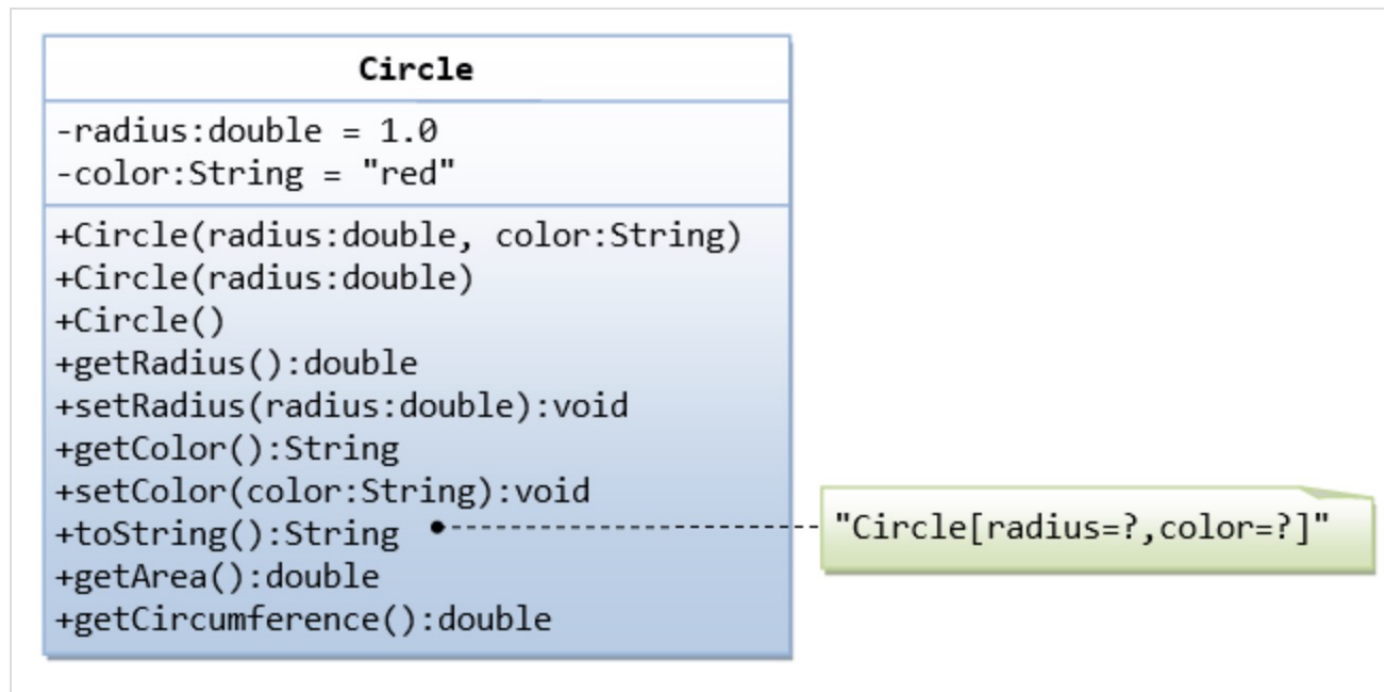
// Setter for radius
public void setRadius(double newRadius) {
    radius = newRadius;
}
```

Từ khóa "this"

- ❖ Là một biến tham chiếu được sử dụng để tham chiếu tới đối tượng của lớp hiện tại.

```
public class Circle {  
    double radius;           // Member variable called "radius"  
    public Circle(double radius) { // Method's argument also called "radius"  
        this.radius = radius;  
        // "radius = radius" does not make sense!  
        // "this.radius" refers to this instance's member variable  
        // "radius" resolved to the method's argument.  
    }  
    ...  
}
```

Sửa lại lớp Circle



Sửa lại lớp Circle (tt)

Circle.java

```
1  /*
2   * The Circle class models a circle with a radius and color.
3   */
4  public class Circle {    // Save as "Circle.java"
5      // The public constants
6      public static final double DEFAULT_RADIUS = 8.8;
7      public static final String DEFAULT_COLOR = "red";
8
9      // The private instance variables
10     private double radius;
11     private String color;
12
13     // The (overloaded) constructors
14     public Circle() {      // 1st (default) Constructor
15         this.radius = DEFAULT_RADIUS;
16         this.color = DEFAULT_COLOR;
17     }
18     public Circle(double radius) {    // 2nd Constructor
19         this.radius = radius;
20         this.color = DEFAULT_COLOR;
21     }
22     public Circle(double radius, String color) { // 3rd Constructor
23         this.radius = radius;
24         this.color = color;
25     }
```

Sửa lại lớp Circle (tt)

```
26
27 // The public getters and setters for the private variables
28 public double getRadius() {
29     return this.radius;
30 }
31 public void setRadius(double radius) {
32     this.radius = radius;
33 }
34 public String getColor() {
35     return this.color;
36 }
37 public void setColor(String color) {
38     this.color = color;
39 }
40
41 // The toString() returns a String description of this instance
42 public String toString() {
43     return "Circle[radius=" + radius + ", color=" + color + "];"
44 }
45
46 // Return the area of this Circle
47 public double getArea() {
48     return radius * radius * Math.PI;
49 }
50
51 // Return the circumference of this Circle
52 public double getCircumference() {
53     return 2.0 * radius * Math.PI;
54 }
55 }
```

Sửa lại lớp Circle (tt)

A Test Driver for the Circle Class

```
// A Test Driver for the Circle class
public class TestCircle {
    public static void main(String[] args) {
        // Test constructors and toString()
        Circle c1 = new Circle(1.1, "blue");
        System.out.println(c1); // toString()
        Circle c2 = new Circle(2.2);
        System.out.println(c2); // toString()
        Circle c3 = new Circle();
        System.out.println(c3); // toString()

        // Test Setters and Getters
        c1.setRadius(2.2);
        c1.setColor("green");
        System.out.println(c1); // toString() to inspect the modified instance
        System.out.println("The radius is: " + c1.getRadius());
        System.out.println("The color is: " + c1.getColor());

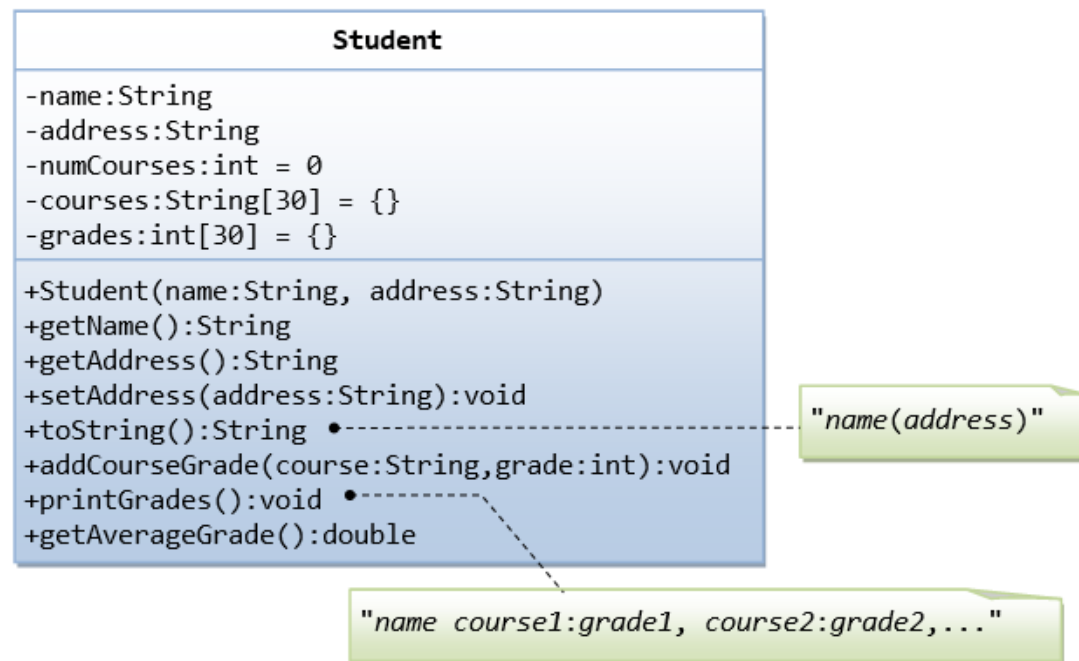
        // Test getArea() and getCircumference()
        System.out.printf("The area is: %.2f\n", c1.getArea());
        System.out.printf("The circumference is: %.2f\n", c1.getCircumference());
    }
}
```

Sửa lại lớp Circle (tt)

The expected outputs are:

```
Circle[radius=1.1, color=blue]
Circle[radius=2.2, color=red]
Circle[radius=8.8, color=red]
Circle[radius=2.2, color=green]
Radius is: 2.2
Color is: green
Area is: 15.21
Circumference is: 13.82
```

Lớp Student



Lớp Student (tt)

The Student Class (Student.java)

```
1  /*
2   * The student class models a student having courses and grades.
3   */
4  public class Student {
5      // The private instance variables
6      private String name;
7      private String address;
8      // The courses taken and grades for the courses are kept in 2 parallel arrays
9      private String[] courses;
10     private int[] grades;    // [0, 100]
11     private int numCourses;  // Number of courses taken so far
12     private static final int MAX_COURSES = 30; // Maximum number of courses taken by student
13
14     // Constructor
15     public Student(String name, String address) {
16         this.name = name;
17         this.address = address;
18         courses = new String[MAX_COURSES]; // allocate arrays
19         grades = new int[MAX_COURSES];
20         numCourses = 0;                    // no courses so far
21     }
22
23     // The public getters and setters.
24     // No setter for name as it is not designed to be changed.
25     public String getName() {
26         return this.name;
27     }
28     public String getAddress() {
29         return this.address;
30     }
31     public void setAddress(String address) {
32         this.address = address;
33     }
```

Lớp Student (tt)

```
34
35 // Describe this instance
36 public String toString() {
37     return name + "(" + address + ")";
38 }
39
40 // Add a course and grade
41 public void addCourseGrade(String course, int grade) {
42     courses[numCourses] = course;
43     grades[numCourses] = grade;
44     ++numCourses;
45 }
46
47 // Print all courses taken and their grades
48 public void printGrades() {
49     System.out.print(name);
50     for (int i = 0; i < numCourses; ++i) {
51         System.out.print(" " + courses[i] + ":" + grades[i]);
52     }
53     System.out.println();
54 }
55
56 // Compute the average grade
57 public double getAverageGrade() {
58     int sum = 0;
59     for (int i = 0; i < numCourses; ++i) {
60         sum += grades[i];
61     }
62     return (double)sum/numCourses;
63 }
64 }
```

Lớp Student (tt)

A Test Driver for the Student Class (TestStudent.java)

```
1  /*
2   * A test driver program for the Student class.
3   */
4  public class TestStudent {
5      public static void main(String[] args) {
6          // Test constructor and toString()
7          Student ahTeck = new Student("Tan Ah Teck", "1 Happy Ave");
8          System.out.println(ahTeck); // toString()
9
10         // Test Setters and Getters
11         ahTeck.setAddress("8 Kg Java");
12         System.out.println(ahTeck); // run toString() to inspect the modified instance
13         System.out.println(ahTeck.getName());
14         System.out.println(ahTeck.getAddress());
15
16         // Test addCourseGrade(), printGrades() and getAverageGrade()
17         ahTeck.addCourseGrade("IM101", 89);
18         ahTeck.addCourseGrade("IM102", 57);
19         ahTeck.addCourseGrade("IM103", 96);
20         ahTeck.printGrades();
21         System.out.printf("The average grade is %.2f%n", ahTeck.getAverageGrade());
22     }
23 }
```

Lớp Student (tt)

The expected outputs are:

```
Tan Ah Teck(1 Happy Ave)
Tan Ah Teck(8 Kg Java)
Tan Ah Teck
8 Kg Java
Tan Ah Teck IM101:89 IM102:57 IM103:96
The average grade is 80.67
```

Hỏi & Đáp

Cảm ơn!